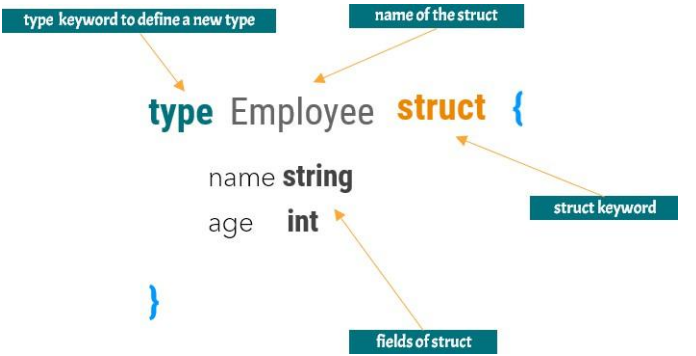
2.7 Структуры

MERION

Структуры (struct) — это пользовательские типы данных, которые позволяют объединять различные поля в одну сущность. Структуры могут содержать поля разных типов.



```go
package main

type Person struct {
    Name string
    Age int
}

func main() {
    alice := Person{Name: "Alice", Age: 30}

    println("Имя:", alice.Name)
    println("Возраст:", alice.Age)
}
```

Изменение полей структуры

```
func main() {
    alice := Person{Name: "Alice",
    Age: 30} alice.Age = 31
    fmt.Println("Обновленный возраст:" ,
    alice.Age)
}
```

MERION


Машина
Двигатель
Колеса

Встраивание в структуры

```
package main

type Person struct { Name string
   Age int
}

type Employee struct { Person
   Position string
}

func main() {
   emp := Employee{
      Person: Person{Name: "Bob", Age: 25},
      Position: "Team Lead",
   }

   println(emp.Name) println(emp.Position)
   println(emp.Age)
}
```

Доступ к методам вложенной структуры внутри одного пакета

```
package main

type Person struct {
    Name string
    age int
}

type Employee struct {
    Person
    Position string
}

func main() {
    emp := Employee{
        Person: Person{Name: "Bob", age: 25},
        Position: "Team Lead",
    }

    println(emp.Name)
    println(emp.Position)
    println(emp.age)
}
```

Доступ к свойствам из другого пакета

```
.
├── main.go
└── types
    └── structs.go
```

```go
package main

func main() {

    emp := Employee{
        Person: Person{Name: "Bob"},
        Position: "Team Lead",
    }

    println(emp.Name)
    println(emp.Position)
    println(emp.age) // ошибка
}
```

MERION

Коллизии атрибутов

```
package main

type Address struct { Name  string Street string
}

type Person struct { Address
    Age int
}

type Company struct { Name  string Location string
}

type Employee struct { Company
    Position string
}

type Combined struct {
  Employee
  Person
}
```

```
func main() {
    comb := Combined{
    Person: Person{
         Address: Address{
         Name:  "John Doe",
            Street: "123 Main St",
         },
         Age: 30,
       },
       Employee: Employee{
       Company: Company{
            Name: "ACME Inc",
            Location: "New York",
         },
         Position: "Engineer",
       },
    }

    println("Person Name:", comb.Name)
}
```

Метод - функция, связанная с определенным типом

```
func (имя_параметра тип_получателя) имя_метода (параметры)
(типы_возвращаемых_результатов){
   тело_метода
}
```

Пример

```
package main
type library []string
func (l library) print(){
    for _, val := range l{
    println(val)
    }
}

func main() {
    lib := library{ "Book1", "Book2", "Book3" }
    lib.print()
}
```

```go
package main

type person struct{
    name string
    age int
}

func (p person) print(){
    println("Имя:", p.name)
    println("Возраст:", p.age)
}

func (p person) eat(meal string){
    println(p.name, "ест", meal)
}

func main() {
    var tom = person { name: "Tom", age: 24 }
    tom.print()
    tom.eat("борщ с капусткой, но не красный" )
}
```

При вызове метода, объект структуры, для которого определен метод, передается в него по значению

```
package main

type person struct{ name
    string
    age int
}
func (p person) updateAge(newAge int){
    p.age = newAge
}

func main() {
    var tom = person { name: "Tom", age: 24 }
    print("before: ", tom.age, " | ")
    tom.updateAge(33)
    println("after: ", tom.age)
}
```

MERION

```go
package main

type person struct{
   name string
   age int
}

func (p *person) updateAge(newAge int){
   p.age = newAge
}

func main() {
   var tom = person { name: "Tom", age: 24 }
    var tomPointer *person = &tom
   print("before: ", tom.age, " | ")
   tomPointer.updateAge(33)
   println("after: ", tom.age)
}
```

MERION